

Deep learning Assignment

Aim:-

To build and train a simple neural network using TensorFlow/Keras to classify handwritten digits 0-9 using the MNIST dataset. Evaluate its performance, visualize training result, Predict handwritten images & study the effect of changing the number of neurons.

Objective:-

1. To load and Explore the MNIST dataset
2. To build and train a neural network using TensorFlow/Keras
3. To Evaluate the model using test accuracy
4. To visualize accuracy & loss during training
5. To predict handwritten digits & compare actual & predicted labels
6. To perform one experiment by modifying the model & compare the results.

1) load and Explore the MNIST dataset & display sample images.

The MNIST dataset can be loaded directly using Tensorflow/Keras. Dataset is divided into training & testing sets. After loading the dataset its dimensions and sample images are explored

code.

→ load and Explore the MNIST dataset.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import from tensorflow import keras
from tensorflow.keras import layers
```

```
# load data (60,000 train + 10,000 test images,
each 28x28 grayscale)
```

```
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.  
load_data()
```

```
Print("Train images:", X_train.shape, "Train labels:",  
y_train.shape)
```

```
Print("Test images:", X_test.shape, "Test labels:",  
y_test.shape)
```

```
Print("Pixel images:", X_train.min(), "to",  
X_train.max())
```

```
Print ("class distribution (train):", np.bincount(y_train))
```

```
# So Display sample Images.
```

```
plt.figure(figsize=(10,4))
```

```
for i in range(10):
```

```
    plt.subplot(2,5, i+1)
```

```
    plt.imshow(X_train[i], cmap="gray")
```

```
    plt.title(f"label: {y_train[i]}")
```

```
    plt.axis("off")
```

```
plt.subtitle ("sample MNIST images")
```

```
plt.tight_layout()
```

```
plt.show()
```

2) Design, compile and train a neural network

```
def build-baseline():
```

```
    model = keras.Sequential([
```

```
        layers.Input(shape=(28,28)),
```

```
        layers.Flatten(),
```

```
        layers.Dense(128, activation="relu"),
```

```
        layers.Dense(10, activation="softmax"),
```

```
    ])
```

```
    model.compile(optimizer="adam",
```

```
                  loss="sparse-categorical-crossentropy",
```

```
                  metrics=["accuracy"])
```

```
return model  
model = build-baseline ()  
model.summary ()
```

```
history = model.fit (X_train, y_test,  
                    epochs = 10,  
                    batch-size = 32,  
                    validation-split = 0.2,  
                    verbose = 1)
```

3) Evaluate the mode using ~~test~~ accuracy

```
test_loss, test_acc = model.evaluate (X_test, y_test,  
                                       verbose = 0)
```

```
Print (f"Test loss : {test_loss : 4f}")
```

```
Print (f"Test accuracy: {test_acc : 4f} ({test_acc * 100 :  
                                         2f} %)")
```

4) Visualize training / validation accuracy & loss

```
def plot-history (h, title = " "):
```

```
    plt.figure(figsize = (12, 4))
```

```
    plt.subplot(1, 2, 1)
```

```
    plt.plot(h.history["accuracy"], label = "Train  
accuracy")
```

```
    plt.plot(h.history["val-accuracy"], label = "validation  
accuracy")
```

```

plt.xlabel("Epoch"); plt.ylabel("Accuracy");
plt.title(f"Accuracy {title}"); plt.legend()

plt.subplot(1, 2, 2)
plt.plot(h.history["loss"], label="Train loss")
plt.plot(h.history["val-loss"], label="Validation
accuracy")
plt.xlabel("Epoch"); plt.ylabel("loss"); plt.title
(f"loss {title}"); plt.legend()

plt.tight_layout(); plt.show()

plt.history(history, "(baseline)")

```

5) Test the model on 5 hand written images and compare the actual & predicted labels.

```

idx = np.random.choice(len(X-test), 5, replace=False)
samples = X-test[idx]
actuals = Y-test[idx]
pred = np.argmax(model.predict(samples, verbose=0), axis=1)
plt.figure(figsize=(10, 3))
for i in range(5):
    plt.subplot(1, 5, i+1)
    plt.imshow(samples[i], cmap="gray")
    color = "green" if pred[i] == actuals[i] else "red"

```

```
color = "green" if pred[i] == actual[i] else "red"
plt.title(f"Actual: {actual[i]} \n Pred: {Pred[i]}", color=
color)
plt.axis("off")
```

```
plt.tight_layout(); plt.show()
```

```
Print("Actual:", actual)
```

```
Print("Prediction:", pred)
```

6) Perform One Experiment by changing either the number of neurons, activation function, number of epochs or adding dropout & briefly compare the results

Adding more neurons + Dropout

```
model 2 = keras.Sequential([
```

```
layers.Input(shape=(28,28)),
```

```
layers.Flatten(),
```

```
layers.Dense(256, activation="relu"),
```

```
layers.Dropout(0.3),
```

```
layers.Dense(10, activation="softmax"),
```

3)

```
model 2.compile(optimizer="adam",
```

```
loss="sparse_categorical_crossentropy",
```

```
metrics=["accuracy"])
```

```
history 2 = model 2.fit(x_train, y_train, epochs = 10,  
                        batch_size = 32
```

```
validation_split = 0.2, verbose = 1)
```

```
plot_history(history 2, "(256 neurons + dropout)")
```

```
loss 2, acc 2 = model 2.evaluate(x_test, y_test, verbose = 0)
```

```
Print(f"{model": < 28} {"Test acc": > 10} {"Test loss": >  
12} {"Final train acc": > 17}")
```

```
f "{history.history['val-accuracy'][-1]: > 15.4f  
{history.history['accuracy'][-1]: > 17.44f}"
```

```
Print(f"{ '256 neurons + Dropout 0.3' : < 28} {acc 2 :  
> 10.4f} {loss 2 : > 12.4f}"
```

```
f "{history 2.history['val-accuracy'][-1]: > 15.4f}  
{history 2.history['accuracy'][-1]: > 17.44f}"
```

Report

Introduction:-

This Assignment is about recognising handwritten numbers using machine learning & Neural Networks. The project uses the MNIST dataset, which contains handwritten digits from 0 to 9.

The main purpose of this project is to train a computer to look at a handwritten number to identify which digit it is. For example, if we give the model an image of a handwritten 5, the model should predict 5.

Objectives:

- * To understand the MNIST handwritten digit dataset
- * To prepare the image data for the model
- * To build a simple neural network
- * To train the model using handwritten digit images
- * To check the accuracy of model
- * To predict digits from test images
- * To test this model with our own handwritten images
- * To compare models with our own handwritten numbers of neurons

Dataset:-

The project uses the MNIST dataset. It contains of handwritten digits from 0 to 9

- * Each image has a size of 28×28 pixels

- * There are 10 different classes
0, 1, 2, 3, 4, 5, 6, 7, 8, 9.

- * The dataset divided into two main parts

 - * Training data

 - * Testing data

some sample images are displayed in the program to understand dataset

Data Preprocessing:

Before giving the images to the neural network the data needs to be processed.

- * The pixel values of the MNIST images are between 0 to 255. The program divides these values by 255 so that they are converted into range between 0 to 1.

- * This process called Normalization.

- * Normalization makes it easier for the neural networks to learn from the images.

Neural Network:

The model contains

Input $\rightarrow$ Flatten $\rightarrow$ Dense(128) $\rightarrow$ Dropout $\rightarrow$ Output(10)

The output layer has 10 neurons, Preprocessing, representing digits 0-9 softmax is used to select the digit with the highest probability.

Training:

The model is trained using

optimizer : Adam

loss : sparse categorical_crossentropy

Epochs : 10

Batch size : 32

validation split : 10%

Evaluation & Prediction:

The trained model is tested using test images

The program calculate loss & accuracy and displays training validation graphs

It also predicts test images & shows the actual & predicted digits.

Testing Handwritten Imager!

Users can upload their own handwritten images. The images are converted to grayscale, resized, centered, normalized & then given to the model for prediction.

Experiment !

A second model using 64 neurons instead of 128 is trained & compared with the original model. This shows how changing the number of neurons can affect model performance.

Technologies Used :-

- * Python
- * Matplotlib, PIL
- * TensorFlow/Keras
- * Google colab
- * Numpy.

Conclusion :-

This Project demonstrates how a neural network can recognize handwritten digits. It covers data preprocessing, model training prediction, Evaluation and comparison of different neural networks structure.